
Train/Test Time Optimizations for Diffusion LLMs: Certainty-Forcing and Early Commit Decoding

Rohan Bansal
rohanbansal@gatech.edu

Krish Katariya
krishkatariya@gatech.edu

Logan Tao
ltao47@gatech.edu

Jason Zhang
jzhang3463@gatech.edu

1 Introduction

Autoregressive (AR) large language models (LLMs) generate text token by token, which has led to impressive generation quality, but naturally requires $O(n)$ steps on n tokens for sequential decoding. Due to this bottleneck, AR-LLMs are limited in inference throughput. Diffusion-based large language models (dLLMs) have emerged as a new and promising alternative that uses *iterative masked denoising*, enabling these models to predict multiple tokens in parallel at each step. However, in practice, current open source dLLMs such as LLaDA-8B still require on the order of the full sequence length in diffusion steps in order to produce high-quality outputs. The core problem here is that certainty tends to converge sequentially in these models, so although many tokens are generally predicted at each diffusion step, the model’s confidence in each token will only pass its confidence threshold in a left-to-right manner.

Unfortunately, this undermines the parallelism advantages of dLLMs, so recent approaches have been addressing this bottleneck. dParallel Chen et al. [2025] improves on this core issue with dLLMs by using a certainty-forcing distillation, where a dLLM learns from a teacher dLLM to not only follow the teacher’s trajectories, but also to achieve a high confidence earlier and across many tokens simultaneously, instead of left to right. This results in faster inference, through a reduction in the number of decoding steps needed.

To further build on this work, we propose applying Prophet [Li et al., 2025] to a dLLM trained using dParallel. Prophet is an inference-time optimization that stops generation early, when the model is very confident on a number of tokens. More specifically, at each generation step Prophet decides whether to continue refining, or to stop and decode all remaining tokens at once based on the average confidence gap between the top-2 candidate tokens. Since dParallel encourages the generation of high-confidence tokens, and Prophet stops early when most tokens are very confident, it is natural to combine the two method together, in order to further reduce the number of decoding steps taken.

Input/Output: Our *input* will be a tokenized prompt. Furthermore, during inference we will make use of the model logits to determine if the model should stop decoding or continue refinement. Our *output* is the completed token sequence after T diffusion steps.

Training Conditions: We follow the training procedure defined by dParallel. *Supervision* follows certainty-forcing distillation for the dLLM (teacher–student). *Architecture* retains the base dLLM (e.g., LLaDA-8B-Instruct). *Optimization* uses the distillation loss for the generator. For hardware consistency, we plan on running all of training on four H100s.

2 Related Work

Diffusion LLMs are a new direction of LLMs that apply iterative denoising (inspired by diffusion models in vision based tasks) for language generation. In some early works such as LLaDA [Nie

et al., 2025] and Dream [Ye et al., 2025], we can see demonstrated viability of this approach, showing that dLLMs have the ability to have accuracy on par with Autoregressive Models. However, these models often end up having as many iterations as the sequence length in order to converge, due to effectively decoding in a blockwise autoregressive fashion. This is what has motivated our method & a surge of research into closing the gap between the theoretical and practical efficiency of dLLMs.

A variety of techniques are being proposed for speeding up diffusion based text generation, such as dLLM-Cache [Liu et al., 2025] which introduces KV caching for bidirectional decoders, storing intermediate key/value representations in order to avoid recomputation across diffusion steps. Fast-dLLM [Wu et al., 2025] proposes splitting generations into different blocks of tokens and treating each block somewhat autoregressively, while generating blocks in parallel. Another paper, AdaBlock-dLLM [Lu et al., 2025] also observes volatility in token confidences across different denoising steps, and as such adapts block sizes accordingly, which will generally help sustain throughput even when confidence fluctuates a lot. In parallel, *dParallel* also introduces a new, parallel decoding method which is trained using certainty-forcing distillation in order to push for earlier convergence of token confidences, which will lead to higher commit rates across positions and will also lower total steps by an order of magnitude without accuracy loss [Chen et al., 2025]. Lastly, *Prophet* is an inference based strategy that uses early commits and by monitoring the confidence gap (using top-2 margin) and, once we find that it is high enough, it will decode all remaining tokens in one shot—cutting refinement steps while maintaining quality [Li et al., 2025].

3 Tentative Technical Approach

3.1 Method Overview

Our method is built based on two complementary advancements in diffusion-based large language models (dLLMs): **dParallel**, which improves training-time efficiency and parallel decoding ability through certainty-forcing distillation, while **Prophet** accelerates inference via *early-commit decoding* based on confidence-gap thresholds.

We will unify these techniques into one **multi-stage certainty-aware decoding pipeline**. While training, using dParallel will teach the model to achieve high token-level certainty earlier and in parallel, which will help transforming sequential convergence into parallel convergence with quick jumps towards high certainty. During inference, Prophet dynamically monitors the model’s average top-2 logit gap to decide when to stop diffusion early, and to decide when to finalize remaining masked tokens in a single parallel pass. Naturally these complement each other towards completing text generation earlier, and our combined approach reduces both training and inference costs.

3.2 Data Processing

To test our method, We use three established benchmarks: **GSM8K** which tests for mathematical reasoning, **MBPP** which tests for code synthesis, and **HumanEval** for functional code generation. Each dataset is drawn directly from its public release without additional preprocessing or augmentation.

- **GSM8K**: Consists of grade-school math word problems. We use the full test split and evaluate generated answers for exact numerical correctness.
- **MBPP**: Contains natural language programming tasks paired with reference solutions. We evaluate each generated program by executing it against the provided unit tests.
- **HumanEval**: Comprises Python function synthesis tasks. For this dataset, models are evaluated on functional correctness using the official execution harness.

3.3 Model / Algorithm Design

Architecture: We use transformer-based dLLMs (LLaDA-8B and Dream-7B) with bidirectional attention and masked-denoising diffusion heads.

Training Stage – dParallel. The teacher model M_{θ_T} generates semi-autoregressive trajectories. The student M_{θ_S} is trained using the combined objective:

$$\mathcal{L}_{CFD} = \mathcal{L}_{Consistency} + \beta \mathcal{L}_{Certainty},$$

where

$$\mathcal{L}_{Consistency} = -\frac{1}{|M_a|} \sum_{i \in M_a} \log p_\theta(y_i | \tilde{Y}),$$

and

$$\mathcal{L}_{Certainty} = \frac{1}{|M_c|} \sum_{i \in M_c} H(p_\theta(v | \tilde{Y}; T)).$$

$\mathcal{L}_{Consistency}$ preserves trajectory alignment with the teacher, while $\mathcal{L}_{Certainty}$ minimizes entropy on correctly predicted tokens to promote high-confidence convergence.

Inference Stage – Prophet. At each diffusion step t , logits L_t are computed, and the mean confidence gap

$$\bar{g}_t = \mathbb{E}_i[L_{t,i}^{(1)} - L_{t,i}^{(2)}]$$

is compared to a dynamic threshold $\tau(p)$ based on decoding progress $p = (T_{max} - t)/T_{max}$:

$$\tau(p) = \begin{cases} \tau_{high}, & p < 0.33 \\ \tau_{mid}, & 0.33 \leq p < 0.67 \\ \tau_{low}, & p \geq 0.67 \end{cases}$$

with $(\tau_{high}, \tau_{mid}, \tau_{low}) = (8, 5, 3)$. If $\bar{g}_t \geq \tau(p)$, early commit decoding is triggered and all remaining masked tokens are filled via $\arg \max(L_t)$.

3.4 Training Details

We will adopt a lightweight fine-tuning setup focused on stability and efficiency rather than extensive hyperparameter exploration, and we use the same diffusion backbone (LLaDA-8B / Dream-7B) to ensure consistency across methods. For our optimizer, we use AdamW due to its general robustness and stable convergence for large scale transformer training. We will use a small learning rate ($1 - 2 \times 10^{-5}$). For batch size and epochs, we will use moderate batch sizes (100s of 2-3 epochs). We can also apply LoRA adapters with rank $r = 8$ to key attention layers, allowing efficient adaptation with minimal memory overhead. For the diffusion schedule, we retain a 50-step diffusion process with a 50% masking ratio, matching standard dLLM setups.

3.5 Rationale / Hypothesis

The hypotheses we intend to test are:

1. For identical prompts and model checkpoints with architecture and temperature held constant, applying Prophet to a dParallel-trained dLLM will *reduce the number of diffusion steps (NFE)* required to reach completion, when compared with using dParallel or Prophet alone.
2. For identical input prompts and evaluation datasets, with the same model checkpoint and holding temperature constant, dParallel+Prophet will *maintain task accuracy* within ± 1 percentage point of the dParallel baseline, showing that early-stopping does not degrade generation quality.
3. The relative reduction in diffusion steps achieved by Prophet will decrease with sequence length. That is, on lower complexity tasks (shorter prompts), dParallel+Prophet will achieve similar task accuracy with a lower NFE when compared to using dParallel or Prophet alone.

3.6 Evaluation Procedure

To ensure a fair comparison, our proposed method will be evaluated against the dParallel and LLaDA-8B-Instruct baselines using an identical experimental setup.

We will use the standard test splits for four benchmarks: GSM8K, MinervaMath, HumanEval, and MBPP. More details about these benchmarks are given in section 4. All models will be evaluated on the exact same set of prompts from these splits.

Accuracy Metrics: (our method must improve)

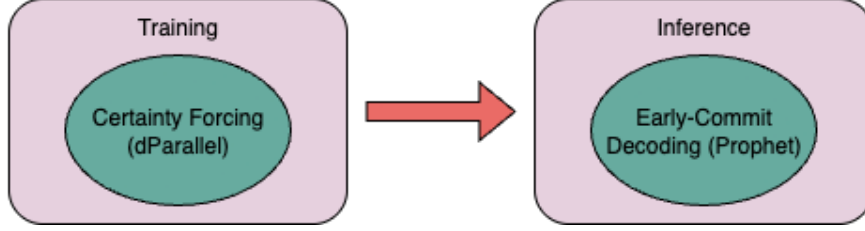


Figure 1: Teaser figure integrating our *current* training and test-time optimizations.

- For the math reasoning tasks (GSM8K, MinervaMath), accuracy will be measured by strict answer matching, where the final numerical answer extracted from the model’s generation must exactly match the ground truth.
- For the code generation tasks (HumanEval, MBPP), we will report pass@1 accuracy. A generation is considered successful if it passes the full set of unit tests associated with each problem.

Efficiency Metrics: (our method must maintain or improve)

- To validate whether we maintain speedups, we will report the total time taken to complete the benchmark tasks on our target hardware (H100) and the NFE (Number of Functional Evaluations). The NFE is directly correlated with the number of diffusion steps, so a low NFE means less diffusion steps to reach an answer.

Other Setup: All models are based on the same architecture (LLaDA-8B-Instruct) to isolate the effect of the improvements. Generation parameters (e.g., sampling temperature, maximum sequence length) will be held constant across all runs.

4 Baseline Results & Trials of Our Method

See our teaser figure at Figure 1.

4.1 Baselines and Benchmarks

We evaluated the following two baselines: dParallel and LLaDA-8B-Instruct. LLaDA-8B-Instruct is the baseline for dParallel, but for our purposes of measuring accuracy instead of speedup, both make good comparisons. All baselines have been evaluated on the following benchmarks: **GSM8K** (grade school math), **MinervaMath** (more difficult math), **HumanEval** (code generation), and **MBPP** (python code generation).

As for our method, we will also evaluate it on the above benchmarks with the same hyperparameters held constant as the baselines, as mentioned earlier.

4.2 Results

We performed experiments as according to 3.6 for our two baselines: **LLaDA-8B-Instruct** and **dParallel**. The key accuracy and efficiency results are summarized in Table 1. We did not yet gather results for the MinervaMath benchmark.

The baseline results confirm that dParallel provides a substantial efficiency improvement over LLaDA-8B-Instruct, with throughput speedups ranging from 2.20x to 5.53x across the evaluated tasks, with accuracy results for both staying the same within a reasonable distribution for all runs.

4.3 Status

Currently, for our method, we are in the process of applying the Prophet [Li et al., 2025] test-time optimization into dParallel, such that we can take full advantage of early-commit decoding. Preliminary empirical results suggest that this optimization is successfully lowering the NFE while

Table 1: Baseline Performance Comparison. Accuracy for GSM8K is measured by exact match; HumanEval and MBPP are measured by pass@1. Throughput is reported in tokens/second.

Benchmark	Metric	Baseline (LLaDA)	dParallel	Improvement
GSM8K	Accuracy (%)	74.68	75.82	+1.14
	Throughput (tok/s)	156.18	864.44	5.53x
HumanEval	Accuracy (%)	31.01	31.51	+0.50
	Throughput (tok/s)	207.81	517.55	2.49x
MBPP	Accuracy (%)	29.40	38.80	+9.40
	Throughput (tok/s)	62.46	137.34	2.20x

keeping the accuracy the same across all datasets. We are encountering arbitrary bugs in the decoding pipeline that we hope to flesh out over the coming week.

As an example of current results: on the prompt "What is the capital of France?", dParallel + Prophet can complete this task in 1 diffusion step (1 NFE), compared to base dParallel which completes this task in 10 diffusion steps (10 NFE).

However, on a more complicated programming task "Write a function to find the similar elements from the given two tuple lists", our method performs 60 more steps than the base dParallel implementation, suggesting badly tuned hyperparameters (the piecewise function defining early-exit thresholds). We plan on improving this function with a continuous version soon, but we can see that our method seems to work better on shorter sequence lengths.

5 Next Steps

Milestones

1. **Finalize Prophet integration:** Complete debugging and integration of the Prophet optimization module into the dParallel decoding step.
2. **Implement improvements over piecewise function:** We noticed that the current test-time optimization currently only uses a piecewise function to determine when to apply early-commit decoding, so we will devise/implement an adaptive method to replace it.
3. **Run Full-Scale Evaluation:** Test on all four benchmarks (GSM8K, MinervaMath, HumanEval, MBPP).
4. **Results & Report:** Compare our method to baselines and write the final report.

Timeline

- **Week 1 (Nov 3 - Nov 9):** Complete Milestone 1 (Prophet integration).
- **Week 2 (Nov 10 - Nov 16):** Milestone 2 (Piecewise improvements).
- **Week 3 (Nov 17 - Nov 23):** Milestone 3 (Run Full Evaluation).
- **Week 4 (Nov 24 - Nov 30):** Milestone 4 (Analysis and Final Report Writing).

Risks & Mitigations A possible risk is that after running the full evaluation of the test-time optimization on dParallel, the results for accuracy and/or speedup are worse; this is possible due to the need to tune the piecewise function that dictates early-commit decoding. To mitigate this, we are planning on implementing an adaptive piecewise or a continuous function instead of a discrete one.

Success Criteria The primary goal is to achieve a moderate to significant accuracy improvement (e.g., +9-10% absolute) over the dParallel baseline on all four benchmarks. The secondary goal is to maintain comparable, if not better throughput (tokens/sec) and NFE reduction relative to dParallel, ensuring our accuracy gains do not come at a significant latency cost.

References

- Zigeng Chen, Gongfan Fang, Xinyin Ma, Ruonan Yu, and Xinchao Wang. dparallel: Learnable parallel decoding for dllms, 2025. URL <https://arxiv.org/abs/2509.26488>.
- Pengxiang Li, Yefan Zhou, Dilxat Muhtar, Lu Yin, Shilin Yan, Li Shen, Yi Liang, Soroush Vosoughi, and Shiwei Liu. Diffusion language models know the answer before decoding, 2025. URL <https://arxiv.org/abs/2508.19982>.
- Zeping Liu, Tianyu Fang, Tianle Chen, et al. dllm-cache: Enabling kv cache and parallel decoding for diffusion llms. *arXiv preprint arXiv:2506.06295*, 2025. URL <https://arxiv.org/abs/2506.06295>.
- Guanxi Lu, Hao Mark Chen, Yuto Karashima, Zhican Wang, Daichi Fujiki, and Hongxiang Fan. Adablock-dllm: Semantic-aware diffusion llm inference via adaptive block size. *arXiv preprint arXiv:2509.26432*, 2025. URL <https://arxiv.org/abs/2509.26432>.
- Weili Nie, Yao Wang, Dinesh Jayaraman, et al. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025. URL <https://arxiv.org/abs/2502.09992>.
- Chengyue Wu, Hao Zhang, Shizhe Diao, et al. Fast-dllm: Training-free acceleration of diffusion llm by enabling kv cache and parallel decoding. *arXiv preprint arXiv:2505.22618*, 2025. URL <https://arxiv.org/abs/2505.22618>.
- Xiangru Ye, Daya Guo, Zhenyu Zhang, et al. Dream 7b: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025. URL <https://arxiv.org/abs/2508.15487>.